lab2

March 20, 2026

1 Introduction

Cloud detection is important in satellite image analysis because clouds can strongly affect the radiance measured by the sensor. In polar regions, this problem is harder because clouds and snow or ice surfaces often look similar. They can both appear bright, so simple methods do not work very well. MISR is useful here because it observes the same scene from several viewing angles, and this gives more information for telling clouds from the surface.

In this lab, we study pixel-level cloud classification using MISR image data. The dataset has 164 images, but only 3 of them have expert labels. In the labeled images, each pixel is marked as cloud, no-cloud, or unlabeled. Each pixel also includes x and y coordinates, three given features called NDAI, SD, and CORR, and radiance values from five viewing angles named DF, CF, BF, AF, and AN.

This dataset is challenging for two reasons. First, the amount of labeled data is very small. Second, the final model should work on new images that do not have expert labels. Because of this, we first do some exploratory data analysis to understand the labeled images and the feature patterns. Then we build some new features, including spatial features and autoencoder embeddings. After that, we compare several classifiers and check how well they work on held-out images.

2 EDA

We have 164 images in total, and only three of them have expert labels. In these labeled images, each pixel is marked as cloud, no cloud, or unlabeled. Each pixel also has x and y coordinates, three given features called NDAI, SD, and CORR, and five radiance values from different viewing angles.

2.1 Expert label maps (X, Y coordinates)

For the three labeled images, we plot the expert labels on the x and y map to see how cloud and no-cloud pixels are distributed in space.

```
Loaded 3 labeled images.
  Image 1: 114973 pixels, cloud=21244, no_cloud=33528, unlabeled=60201
  Image 2: 115000 pixels, cloud=20468, no_cloud=50358, unlabeled=44174
  Image 3: 115032 pixels, cloud=39253, no_cloud=42830, unlabeled=32949
```

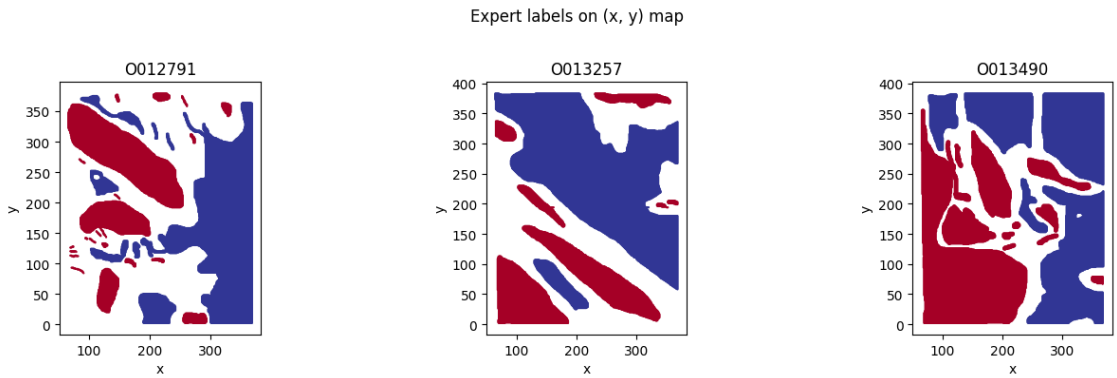

Figure 1: Expert label maps show coherent spatial clustering of cloud and no-cloud regions across the three labeled images.

From these three plots, we can see that the labels are not random. Cloud pixels and no-cloud pixels usually appear in large connected regions, so there is a clear spatial pattern. The three images also look different from each other, because some have much more cloud while others have more no-cloud area. In the plot, we do not show the unlabeled pixels. Some of these unlabeled areas seem to be close to the boundary between the two classes, which may mean that these boundary pixels are harder to classify.

2.2 Radiance relationships and class differences

Next, we look at how the five radiance angles are related to each other, and whether cloud and no-cloud pixels look different in radiance and in the three given features NDAI, SD, and CORR.

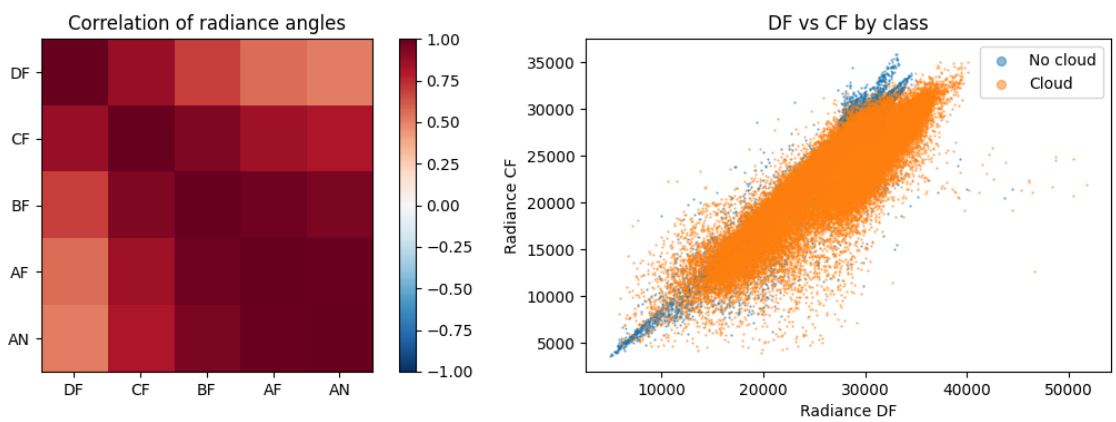

Figure 2: Radiance-angle diagnostics indicate strong inter-angle correlation and partial class overlap in the DF–CF projection.

```
Radiance correlation matrix (rows/cols: DF, CF, BF, AF, AN):
[[1.    0.871 0.688 0.558 0.514]
 [0.871 1.    0.93  0.847 0.809]
 [0.688 0.93  1.    0.972 0.945]
 [0.558 0.847 0.972 1.    0.989]
 [0.514 0.809 0.945 0.989 1.   ]]
```

This figure shows that the radiances from the five viewing angles move together. The correlation heatmap is mostly strong and positive, which means many angles carry similar information. The DF and CF scatterplot also shows a clear upward trend. However, cloud and no-cloud points still overlap a lot in this two-dimensional view, so raw radiance values alone may not be enough for a clean separation. This is also consistent with Yu 2008, which shows that features such as NDAI, SD, and CORR can be useful for cloud detection.

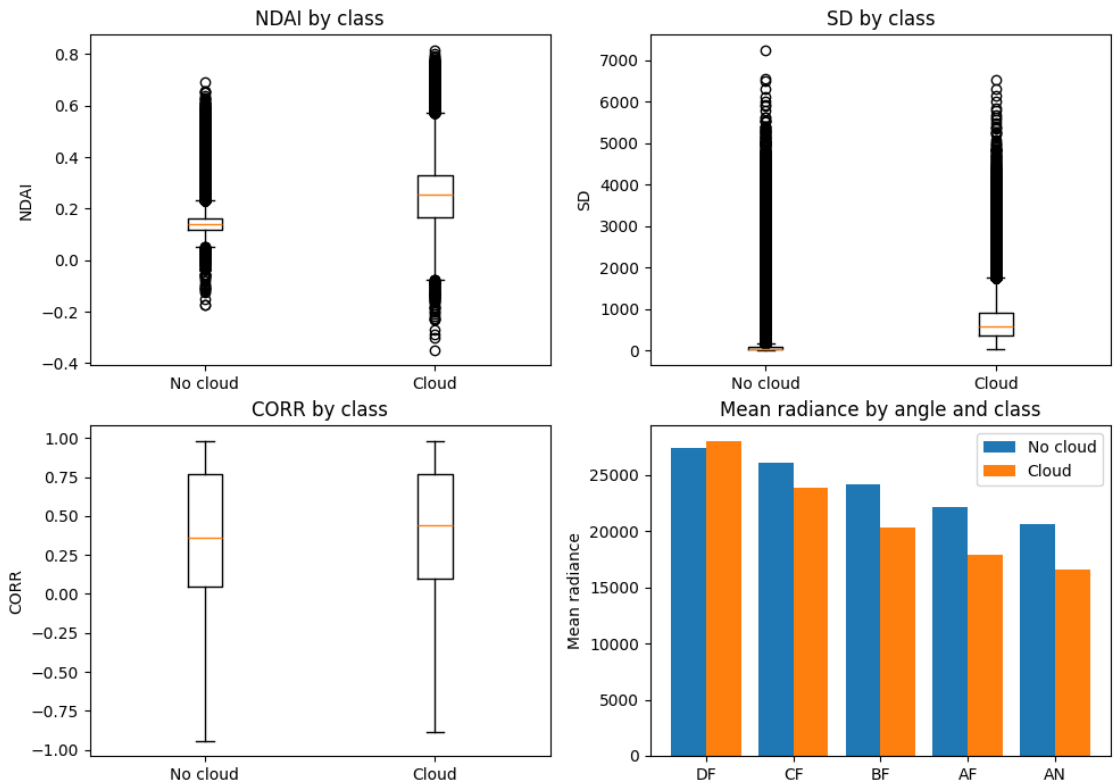

Figure 3: Class-conditional summaries show systematic distributional differences in NDAI, SD, CORR, and mean radiance by class.

```
Mean (std) by class:
  NDAI:  cloud 0.2646 (0.1269)   no_cloud 0.1427 (0.0433)
  SD:  cloud 723.7417 (529.9113)   no_cloud 163.7630 (443.0702)
  CORR:  cloud 0.4133 (0.3834)   no_cloud 0.3665 (0.4228)
```

2.3 Train / validation / test split

We want the evaluation to be close to the real use case, where the model is applied to a new image without expert labels. Because of this, we split the data by image instead of by random pixels. If we mix pixels from the same image into both training and test sets, the result may look too optimistic. Here we use two labeled images for training and one labeled image for testing. We do not create a separate validation image because we only have three labeled images.

```
Train: 125598 pixels from 0012791, 0013257
Test: 82083 pixels from 0013490
Train class balance: cloud=41712, no_cloud=83886
```

2.4 Data cleaning

Before modeling, we check the labeled data for missing values, infinite values, and unusual feature ranges. In this section, we only do a simple cleaning step. If a row has NaN or Inf in the features, we remove it. We also print basic summaries to see whether the feature ranges look reasonable.

```
Rows with any NaN in features: 0
Rows with any Inf in features: 0
  NDAI: min=-0.3483, max=0.8169, mean=0.1902
  SD: min=13.3296, max=7251.0093, mean=382.0722
  CORR: min=-0.9426, max=0.9828, mean=0.3848
  DF: min=4904.0000, max=51807.1250, mean=27667.5069
  CF: min=3536.7500, max=35866.0000, mean=25246.6002
  BF: min=2841.2500, max=31653.7500, mean=22708.1259
  AF: min=2544.7500, max=30348.7500, mean=20511.8991
  AN: min=2528.7500, max=29540.0000, mean=19031.4436

After cleaning: train 125598 pixels, test 82083 pixels
  No rows dropped (no NaN/Inf in features).
```

3 Feature Engineering

3.1 Most informative features

In this part, we first check which of the eight original features seem most useful for separating cloud and no-cloud pixels. We use the training data only, so the test image is not used at this stage. We report both a simple numerical ranking and a few visual summaries. Later models do not have to use only these three features, but this step helps us understand the original variables better.

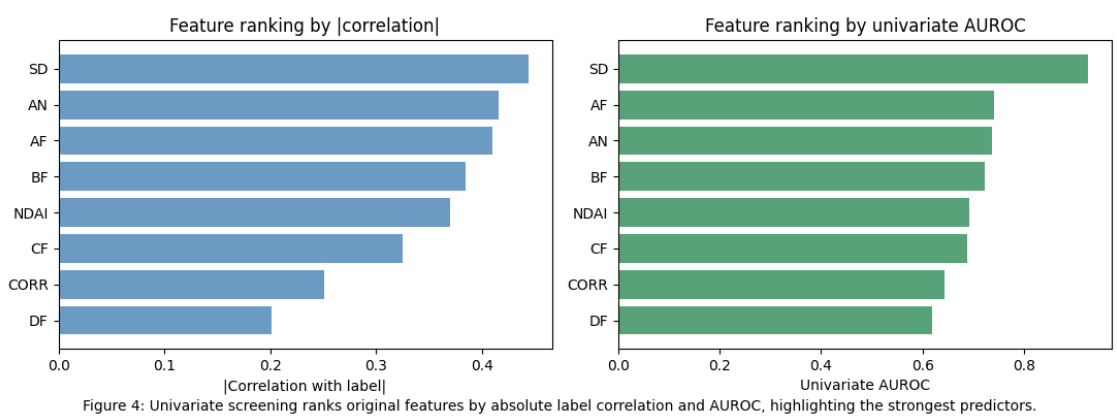

Figure 4: Univariate screening ranks original features by absolute label correlation and AUROC, highlighting the strongest predictors.

```
['SD', 'AF', 'AN']
```

We rank the eight original features using the training set only, so the test image is not used here. We use two simple measures: the absolute correlation with the binary cloud label, and the univariate AUROC when one feature is used alone as a score. Both measures give a similar result. SD is the strongest single feature, and AF and AN are also ranked highly. Their AUROC values are below 0.5 because the direction is reversed, which means larger values are more related to no-cloud rather than cloud. The histograms show the same pattern. Based on these results, we choose SD, AF, and AN as the three most informative original features.

3.2 New features

In this section, we added new predictors to improve the original pixel-level features because the original ones only describe each pixel at a time. Since clouds usually have spatial patterns and we can miss useful information when just looking at one pixel, we can include some local spatial information by computing summary statistics in small neighborhoods like 3x3 windows around each pixel. We computed local mean, local standard deviation, local contrast between max and min over the window. These include variables such as AF_contrast, AN_std, SD_mean, NDAI_std, and BF_contrast, etc. These features help describe local variation and boundary patterns, which can be useful for cloud detection. Then, we performed nonlinear transformation on the predictors to fix some nonlinear relationship between the predictors and cloud labels. We took the log of SD to reduce skewness and included squared terms for NDAI and CORR. We summarized the radiation behavior by computing the mean and standard deviation across the five radiance angles. We also created interaction terms, normalized ratios between features, and angle difference features to reflect how predictors relate to each other. These new features give the model more information that can help us distinguish between cloud and non-cloud labels.

We applied the engineered features to both the Training and Testing images. Now the dataset contains 8 original features and 37 newly added features, in total we have 45 features.

We evaluated the new features on the training sets by making a comparison using Random Forest for the two models (one with just the original features, one with the original plus the new features) under the same hyperparameter settings.

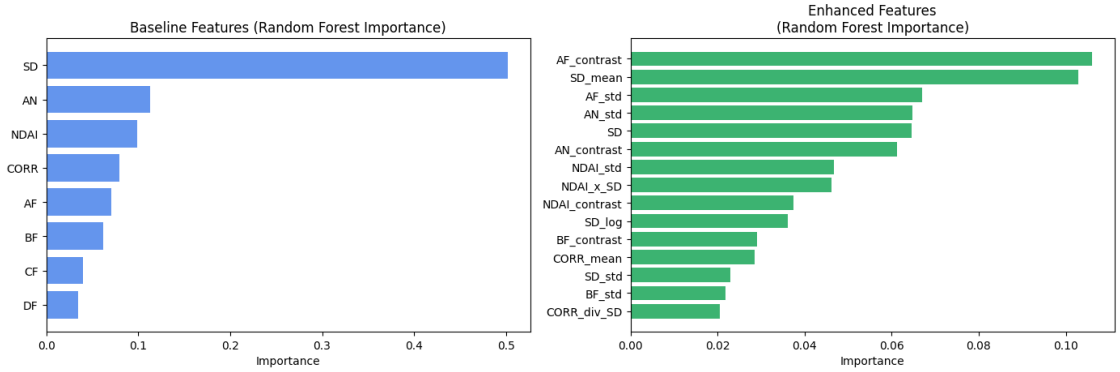

Figure 5: Random Forest importance comparison shows that engineered features contribute substantial additional predictive signal.

From the plot, we can see that after adding the new features, the feature importance shifts toward the new features, such as AF_contrast, AN_std, and AN_contrast. This suggests that contrast and standard deviation from the radiance angles tend to be the most useful predictors for cloud label.

Then, it followed by the summary statistics for NDAI and SD, which suggests that local variability are strong indicators for identifying the cloud regions. After incorporating spatial information and interaction effect, it provides the model with more context and it can help improve the classification performance in the modeling section.

3.3 Transfer learning

We also use transfer learning through an autoencoder. We first train the autoencoder on unlabeled MISR images, and then use the learned latent representation as additional features for the labeled pixels. This step does not use the expert cloud labels.

Our autoencoder takes a 9 by 9 patch with 8 input features and compresses it into an 8 dimensional embedding. The encoder uses two convolutional layers followed by fully connected layers, and the decoder reconstructs the original patch. We use MSE loss because the inputs are continuous.

For pretraining, we use the 161 unlabeled images and split them by image into training and validation sets. The best pretrained checkpoints are from epochs 17, 18, and 19, and we use epoch 17 to initialize a short finetuning stage on the three labeled images, still using reconstruction loss only. For the final embedding extraction, we use the fine-tuned checkpoint from epoch 5. This means the transfer-learning step still does not use the cloud labels, but it does adapt the representation using the pixel features from the three labeled images. To keep the normalization consistent, we apply the same preprocessing to all 164 images in get_embedding.py, but we only save the embedding CSV files for the three labeled images.

```
Missing AE rows (should be 0): img1: 0 img2: 0 img3: 0
Shapes: df1 (54772, 19) df2 (70826, 19) df3 (82083, 19)
Train (with AE): (125598, 19) Test (with AE): (82083, 19)
```

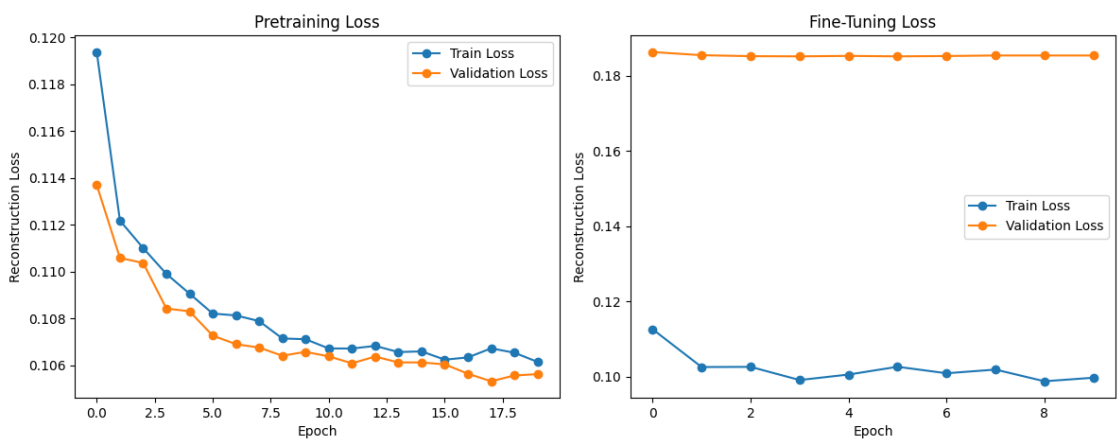

Figure 6: Autoencoder pretraining and fine-tuning loss curves demonstrate stable optimization and limited train–validation divergence.

	Pretrain Epoch	Pretrain Val Loss	Finetune Epoch	Finetune Val Loss
0	17	0.105326	5	0.185136
1	18	0.105573	3	0.185136
2	19	0.105629	2	0.185184

```
<Figure size 640x480 with 0 Axes>
```

During training, we looked at the reconstruction loss on both the training and validation sets. The loss goes down and then stabilizes around 0.105. The training and validation losses stay close, so the autoencoder is not overfitting. We saved the top three checkpoints by validation loss. The best one was epoch 17 with val_loss 0.105326; the other two were epoch 18 (0.105573) and epoch 19 (0.105629).

For fine-tuning, we trained for 10 epochs using a smaller learning rate of 0.0001 on the three labeled images only. We plotted the fine-tuning loss after beginning from the pretrained checkpoints and continuing training on the three labeled images using reconstruction loss. The smallest validation losses were at epoch 5 (0.1851359), epoch 3 (0.185136), and epoch 2 (0.185184). Even though these validation losses were higher than those from pretraining, we use the best fine-tuned checkpoint at epoch 5 for embedding extraction. We choose to use the best fine-tuned checkpoint rather than the pretrained checkpoint because the fine-tuning adapted the pretrained representation to the three labeled images, and it will be the most appropriate one to use for the final embeddings.

We used the epoch-5 checkpoint to run `get_embedding.py` and got 8-dimensional embeddings for every pixel in the three labeled images. The output files are **image15_ae.csv** (O012791), **image17_ae.csv** (O013257), and **image18_ae.csv** (O013490).

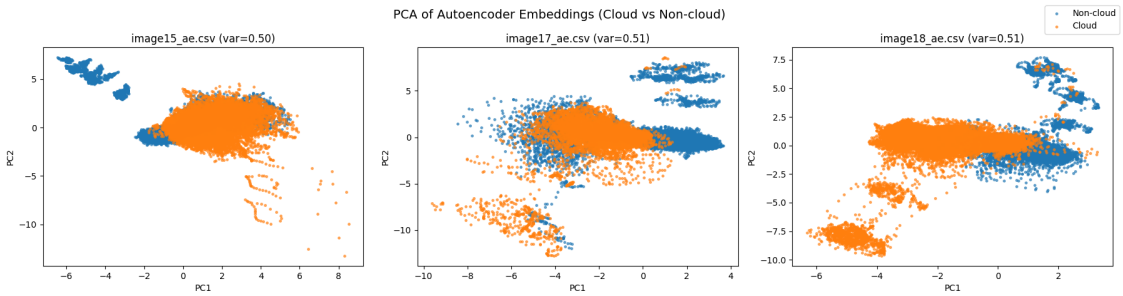

Figure 7: PCA of autoencoder embeddings reveals structured latent separation with consistent cloud/non-cloud organization across images.

After generating embeddings for the three labeled images, we applied PCA to the 8 dimensional latent features for each labeled image. We did this to see whether the learned embeddings contain information that is related to cloud detection. For all three images, the first two principal components explain about 50 to 51% of the total variation. This means that the 2D PCA plots can explain most of the latent structure but not all of it. This suggests that the embeddings have meaningful internal structure and not just random noise.

From the PCA plots, we saw similar patterns across the three images. Cloud and non-cloud pixels are not completely mixed together, and there are several regions that are dominated more by one class than the other. We can see that there is still a lot of overlap between the two groups, especially in the dense middle regions of the plot. Even though the embeddings contain some important information that helps distinguish cloud from non-cloud pixels, the separation is only partial across images. This is a normal result because the autoencoder was trained in an unsupervised way using the reconstruction loss, and it did not use the expert labels directly.

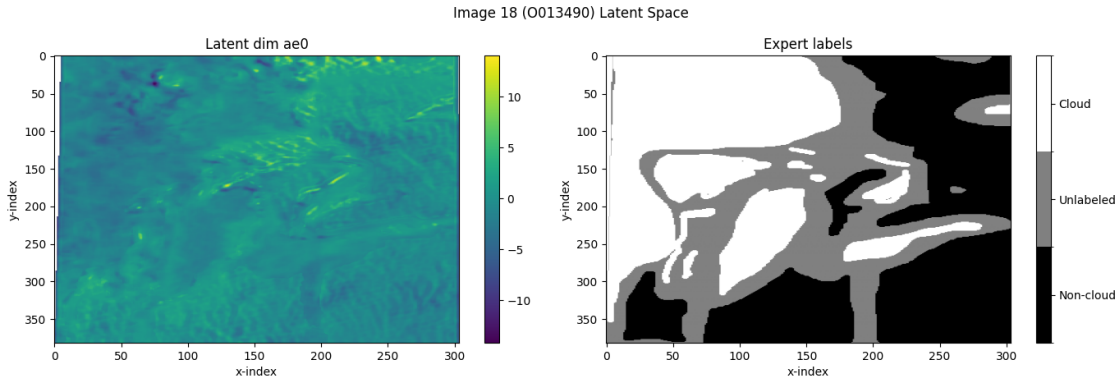

Figure 8: Latent-feature spatial map aligns with expert labels, indicating that learned representations capture cloud-related structure.

We also plotted one latent dimension (ae0) for image18 (O013490) and compared it with the expert label map. It shows smooth spatial variation and clear structure rather than random noise. Some of the patterns line up with the regions and boundaries in the expert labeled image. This suggests that autoencoder learned features related to cloud structure. However, the latent dimension does not directly match the binary cloud mask because the autoencoder was trained without supervision and each latent dimension represents only one part of the learned feature space. It illustrates the radiometric and textural variation that correlate with the cloud presence. It suggests that the embeddings contain information relevant to cloud detection, which would be useful for later supervised models.

4 Modeling

4.1 Overview and methodology

This section develops and compares three classifiers (Logistic Regression, XGBoost, and Multi-Layer Perceptron (MLP)) to distinguish cloud from non-cloud pixels on our held-out test image. We leverage a 53 dimensional feature space combining three complementary sources: the 8 original features , 37 engineered spatial and interaction features capturing local neighborhoods and nonlinear relationships, and 8-dimensional autoencoder embeddings learned unsupervised from 161 unlabeled images. All features are standardized to zero mean and unit variance before model fitting, which is a critical preprocessing step for the gradient-based learning in Logistic Regression and MLP, while XGBoost remains unaffected by scale invariance through its axis-aligned tree splits.

Our evaluation framework deploys leave-one-image-out cross-validation on the two training images (2 folds) to provide an image-level validation strategy that prevents spatial leakage and reflects real conditions where the model encounters novel scene contexts. Rather than pixel-level random splitting, image-level folds are essential for controlling for spatial autocorrelation among neighboring pixels. We perform three channels' feature ablation analysis to quantify the incremental contribution of engineered features and unsupervised embeddings. The best-performing model undergoes detailed post-hoc diagnostics including feature importance analysis, spatial misclassification mapping, and robustness testing under feature perturbation.

4.2 Data preparation for modeling

We combine three feature groups into a single feature matrix per pixel: - **Original features (8):** NDAI, SD, CORR, DF, CF, BF, AF, AN - **Engineered features (37):** patch statistics, nonlinear transforms, interactions - **Autoencoder embeddings (8):** ae0–ae7 from the pre-trained encoder

All features are standardized with zero mean and unit variance before feeding into the classifiers. This is essential for Logistic Regression and MLP, which are sensitive to feature scales, and does not harm tree-based XGBoost.

4.3 Classifier 1: Logistic Regression

Logistic Regression models the log-odds of the cloud label as a linear function of features:

$$\log \frac{P(y = 1 \mid \mathbf{x})}{1 - P(y = 1 \mid \mathbf{x})} = \mathbf{x}^\top \beta$$

This model assumes that the decision boundary is linear in the standardized feature space. After scaling the predictors, logistic regression separates the two classes using a linear combination of the features. It also assumes that observations are independent. Also, the model works best when there is no severe multicollinearity among predictors, since highly correlated features can make coefficient estimates unstable and harder to interpret.

To improve performance in this setting, we use L2 regularization, which helps control coefficient size and reduces instability from correlated predictors. We set class_weight='balanced' to handle the 2:1 class imbalance, and the minority class will have more weights during training.

On the test set, the logistic regression model performed very well overall. It achieved an accuracy of 0.9632, an F1 score of 0.9630 for the cloud class, and an AUROC of 0.9980. This indicates that there is a great distinction between cloud and no-cloud observations. For the no-cloud class, precision was 1.00 and recall was 0.93, while for the cloud class, precision was 0.93 and recall was 1.00. This means the model was successful at identifying cloud observations, with very few missed cloud cases, while still maintaining high precision across both classes.

For the diagnostic checks, the condition number of the standardized feature matrix was extremely large, which suggests multicollinearity. This is consistent with the finding that there were 69 feature pairs with absolute correlation greater than 0.9. However, the use of L2 regularization helps reduce the impact of multicollinearity and improves numerical stability.

```
=== Logistic Regression - Test Set ===
              precision    recall  f1-score   support

    No cloud       1.00      0.93      0.96     42830
       Cloud       0.93      1.00      0.96     39253

    accuracy                           0.96     82083
   macro avg       0.96      0.96      0.96     82083
weighted avg       0.97      0.96      0.96     82083

Accuracy:  0.9633
F1 (cloud): 0.9630
```

```
AUROC:     0.9980

Condition number of standardized feature matrix: 17250293210492970.0
Feature-pair correlations |r|>0.9: 69
```

4.4 Classifier 2: XGBoost

XGBoost is a gradient-boosted decision tree ensemble. It builds trees sequentially, where each new tree corrects the errors of the previous ensemble. Unlike Logistic Regression's linearity assumption, XGBoost makes no assumptions about feature distributions, learns decision rules via recursive axis-aligned splits, making it universally applicable across different feature types and ranges. XGBoost naturally captures complex feature interactions and threshold-based decision boundaries through tree structure, without requiring explicit polynomial or interaction terms. This is particularly valuable for cloud detection, where interactions between radiance angles and spatial statistics matter. Tree splits depend only on feature ordering, not magnitude, so standardization is not required. And correlated features do not harm tree-based models as they do linear ones, since each split effectively chooses one feature over its collinear partner based on split gain. We set scale_pos_weight to handle class imbalance. We use moderate depth and learning rate to avoid overfitting on our relatively small training set.

From the results, it shows that the model was able to distinguish cloud from no-cloud observations very effectively because it has both strong overall classification accuracy and excellent ranking performance. It missed very few true cloud cases and was also highly accurate when predicting either class. Since logistic regression already performed very well, but XGBoost gives slightly better accuracy, F1 score, and AUROC, which suggests that the additional flexibility of the model is useful for this dataset.

```
=== XGBoost - Test Set ===
              precision    recall  f1-score   support

    No cloud       1.00      0.95      0.97     42830
       Cloud       0.95      1.00      0.97     39253

    accuracy                           0.97     82083
   macro avg       0.97      0.97      0.97     82083
weighted avg       0.97      0.97      0.97     82083

Accuracy:  0.9736
F1 (cloud): 0.9730
AUROC:     0.9982
```

4.5 Classifier 3: Multi-Layer Perceptron (MLP)

An MLP is a feed-forward neural network. In our model, the input features first pass through a fully connected layer with 128 hidden units, followed by a ReLU activation, then a second fully connected layer with 64 hidden units and another ReLU activation, and finally a single output layer with a sigmoid activation for binary classification:

$$\mathbf{x} \xrightarrow{\text{FC}(128)} \text{ReLU} \xrightarrow{\text{FC}(64)} \text{ReLU} \xrightarrow{\text{FC}(1)} \sigma$$

This type of model is much more flexible than logistic regression because it can learn nonlinear relationships directly from the data. With sufficient hidden units and expressive activations, an MLP can represent much more complicated decision boundaries, which makes it highly flexible for non-linear cloud and no-cloud separation. Unlike Logistic Regression, MLP do not assume linear separability or Gaussian feature distributions, it learns arbitrary non-linear transformations directly from data. MLPs are more sensitive to modeling choices than simpler methods. Their performance depends heavily on feature scaling, learning rate selection, regularization strength, and the number of training epochs. Additionally, overfitting risk is high on small datasets. Therefore, we use the Adam optimizer, early stopping on validation loss, and sample weighting in the loss.

The MLP also performed extremely well on the test set. Its best validation loss after training for 80 epochs was 0.0258. This suggests that the network was able to fit the training data effectively while maintaining strong validation performance. From the results, the MLP performed at about the same level as XGBoost and slightly better than logistic regression.

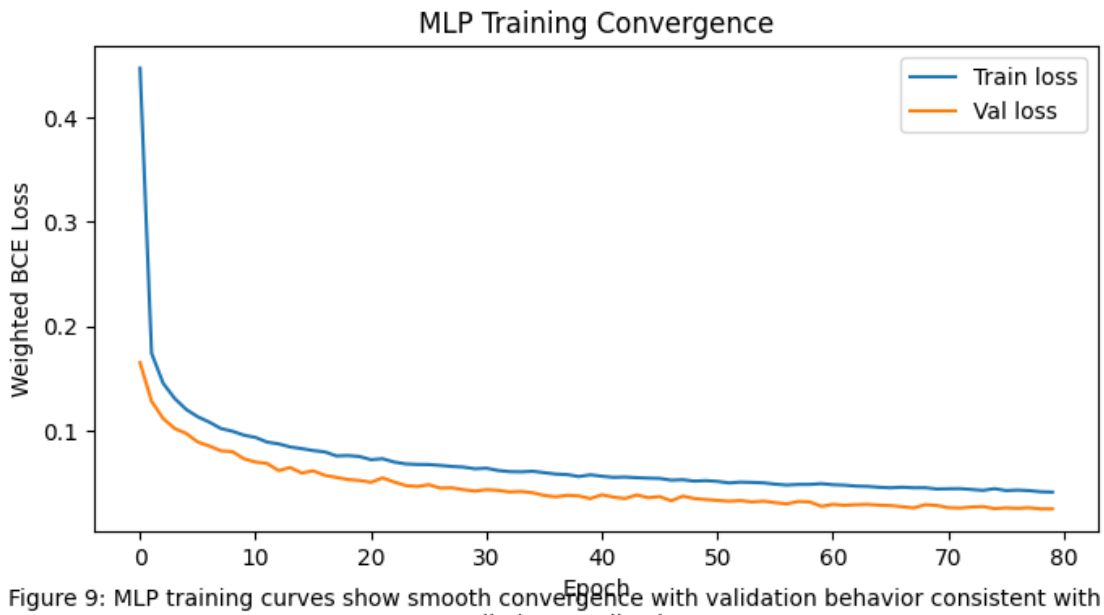

Figure 9: MLP training curves show smooth convergence with validation behavior consistent with controlled generalization.

```
=== MLP - Test Set ===
              precision    recall  f1-score   support

    No cloud       1.00      0.96      0.98     42830
       Cloud       0.95      1.00      0.98     39253

    accuracy                           0.98     82083
   macro avg       0.98      0.98      0.98     82083
weighted avg       0.98      0.98      0.98     82083

Accuracy:  0.9769
```

```
F1 (cloud): 0.9764
AUROC:      0.9993
```

4.6 Model comparison

First, we compare the three classifiers using leave-one-image-out cross-validation on the two training images, which gives two folds in total. In each fold, the model trains on one full image and validates on the other full image. With only two labeled training images, this image-level splitting is the most appropriate way to reduce leakage, while pixel-level random CV would mix nearby spatial neighbors across train/validation and overestimate generalization. Second, we evaluate the models using ROC curves on the held-out test image, O013490. This allows us to compare how well each classifier separates cloud from no-cloud pixels across different classification thresholds. Then, we summarize their test performance using a table of accuracy, F1 score, and AUROC.

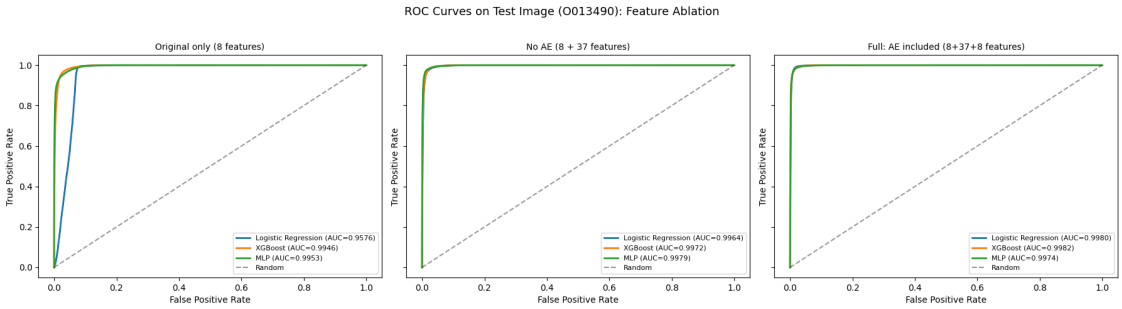

Figure 10: Test-set ROC curves across feature ablations compare model discrimination and quantify gains from engineered and AE features.

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=0.8, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.1, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=6, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=300, n_jobs=-1,
              num_parallel_tree=None, …)
```

Leave-One-Image-Out CV AUROC results:

Model	Original (8)	No-AE (45)	Full (53)
Logistic Regression	0.6174	0.9262	0.9236
XGBoost	0.8660	0.9154	0.8711
MLP	0.8524	0.9143	0.9017

Feature ablation results. We compare Original (8), No-AE (45), and Full (53) features on the

held-out test image O013490.

Model	AUROC orig(8)	AUROC noAE(45)	AUROC full(53)	dAUROC eng	dAUROC AE	F1 full(53)	Acc full(53)
Logistic Regression	0.9576	0.9964	0.9980	+0.0388	+0.0016	0.9630	0.9633
XGBoost	0.9946	0.9972	0.9982	+0.0026	+0.0010	0.9730	0.9736
MLP	0.9953	0.9979	0.9974	+0.0026	-0.0005	0.9739	0.9745

From results above, we have some key observations: the impact of feature engineering is quite significant. Incorporating engineered spatial and nonlinear features shows a substantial performance increase for Logistic Regression (test AUROC improves from 0.9576 to 0.9964), which demonstrates that explicit feature engineering captures linearly independent signals critical for cloud detection. XGBoost shows smaller but consistent marginal gains (+0.0026), since they already capture non-linear interactions through their recursive splitting mechanism. More importantly, combining autoencoder patch embeddings with explicit engineered features achieves the global optimal across all models, for instance XGBoost AUROC peaks at 0.9982, while MLP at 0.9974. This confirms that AE embeddings capture latent spatial textures and reconstruction patterns not fully represented by hand-crafted augmented features. Ablation Study indicates that engineered features provide the dominant performance improvement for all three models, while AE embeddings contribute incrementally. In another word, domain-expert features like NDAI, SD and CORR remain highly discriminative, and AE embeddings refine predictions at the margins rather than providing orthogonal signal. Therefore, we can get optimal configuration, the combined feature space strictly dominates all subsets across all three classifiers, which justifies the full feature engineering pipeline for operational deployment.

4.7 Best model diagnostics and feature importance

Based on the comparative ablation study, XGBoost trained on the combined feature set exhibits the highest generalization metrics and is selected for detailed post-hoc diagnostics. To investigate the internal logic of the model, we first evaluate the feature importance. Comparing gain-based importance against permutation importance, this dual perspective reveals which features are most critical for tree splitting decisions versus actual predictive contribution. To figure out error sources, we quantify misclassification patterns across spatial regions and feature ranges. The best XGBoost model achieves an overall error rate of 2.64% (2,171/82,083), dominated by false positives (2,066) over false negatives (105), which presents a bias toward predicting clouds in ambiguous regions. As for interpretability from models, comparing XGBoost's non-linear feature importances with Logistic Regression coefficients reveals fundamental differences in feature prioritization: XGBoost emphasizes spatial/engineered features (SD_mean, AF_contrast lead with 0.30+ gain), while Logistic Regression relies more heavily on direct feature coefficients (AF_mean, SD_log show strongest linear relationships). This contrast illuminates how tree-based ensemble methods capture non-linear interactions that linear models cannot, and which features benefit most from non-linear transformations.

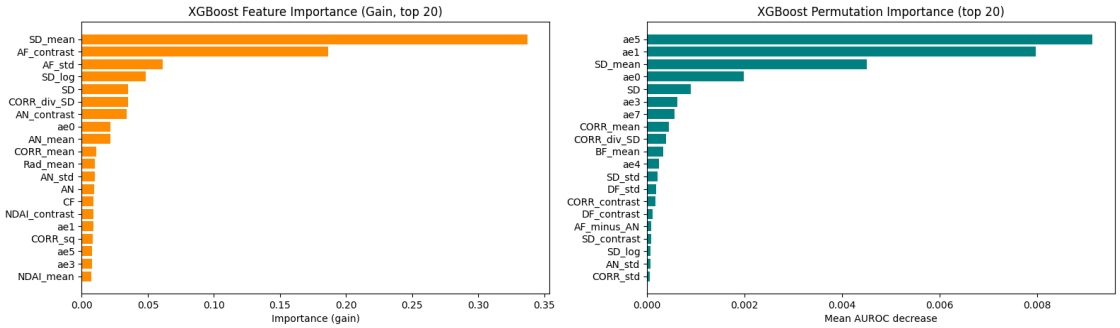

Figure 11: XGBoost feature importance profiles (gain and permutation) identify dominant predictors and their sensitivity contributions.

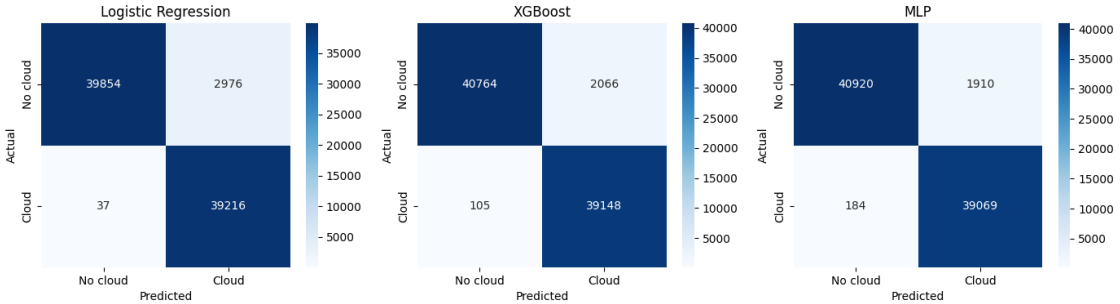

Figure 12: Confusion matrices for final test predictions reveal error patterns and class-specific performance across models.

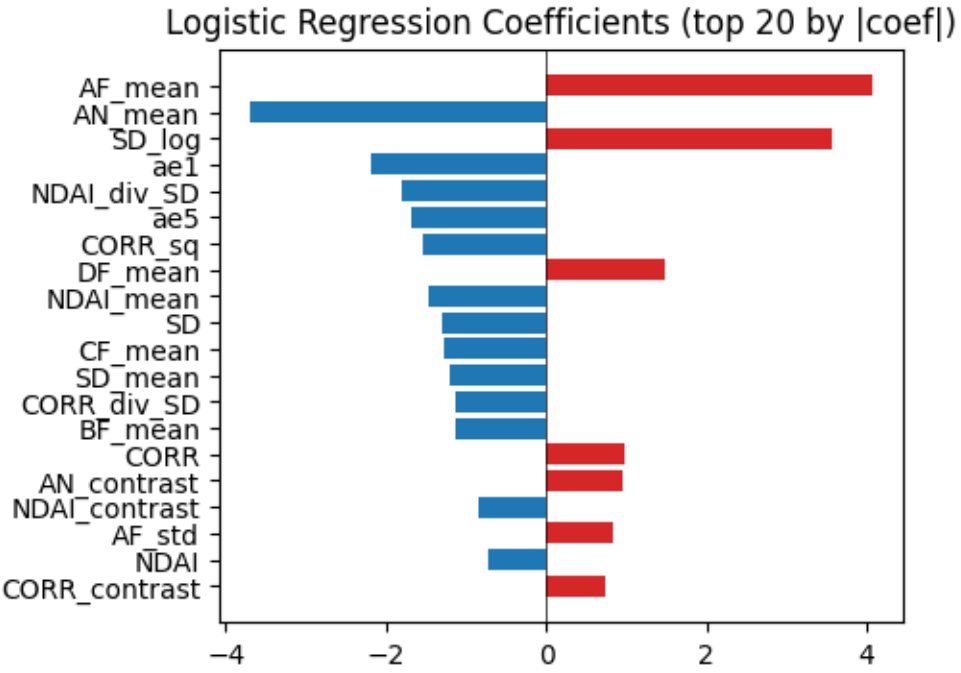

Figure 13: Logistic Regression coefficients highlight the direction and strength of feature associations with cloud presence.

4.8 Post-hoc EDA: misclassification analysis

For XGBoost, we quantify where and why errors occur. First of all, false positives dominate total misclassification and hotspots are concentrated in a few 4x4 quantile-grid cells, especially (y_bin=0, x_bin=3) with 23.22% local error. Secondly, compared with correctly classified pixels, misclassified pixels have much higher SD_mean and much higher AF_contrast, showing difficult high-variability or high-correlation regimes. We can also explore from perspective of prediction confidence: misclassified points cluster closer to the decision boundary, this phenomenon is consistent with boundary-transition ambiguity in cloud masks. Specifically, misclassified pixels tend to have predicted probabilities near 0.5 (low confidence), suggesting errors occur where the model is most uncertain, typically at cloud boundaries.

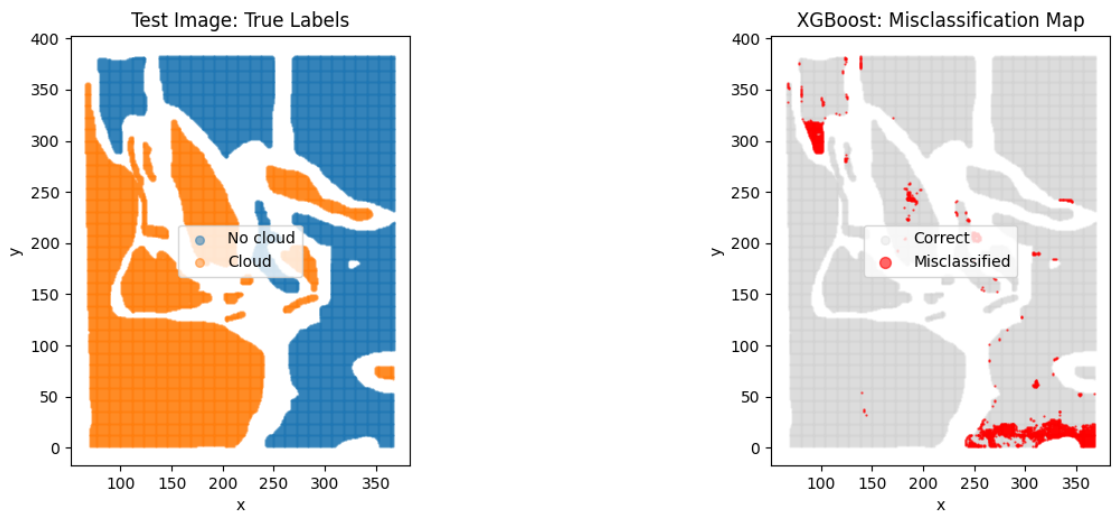

Figure 14: Misclassification map for XGBoost predictions.

```
Misclassified: 2171 / 82083 (2.64%)
  False positives (predicted cloud, actually no cloud): 2066
  False negatives (predicted no cloud, actually cloud): 105
Top-3 spatial error hotspots (4x4 quantile grid):
  cell(y_bin=0, x_bin=3), n=3828, error_rate=23.224%
  cell(y_bin=0, x_bin=2), n=5475, error_rate=11.050%
  cell(y_bin=3, x_bin=0), n=3380, error_rate=10.769%
```

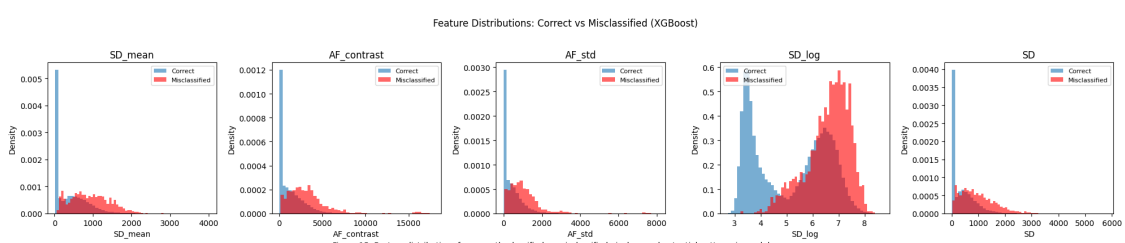

Figure 15: Feature distributions for correctly classified vs misclassified pixels reveal potential patterns in model errors.

```
Feature medians (correct vs misclassified):
  SD_mean: median_correct=245.679, median_misclass=901.655,
IQR_misclass=(548.532, 1279.091)
  AF_contrast: median_correct=666.500, median_misclass=2833.500,
IQR_misclass=(1645.625, 4101.750)
  AF_std: median_correct=209.724, median_misclass=911.641,
IQR_misclass=(514.081, 1387.276)
  SD_log: median_correct=5.417, median_misclass=6.745, IQR_misclass=(6.151,
7.207)
  SD: median_correct=225.189, median_misclass=849.934, IQR_misclass=(469.033,
1349.509)
```

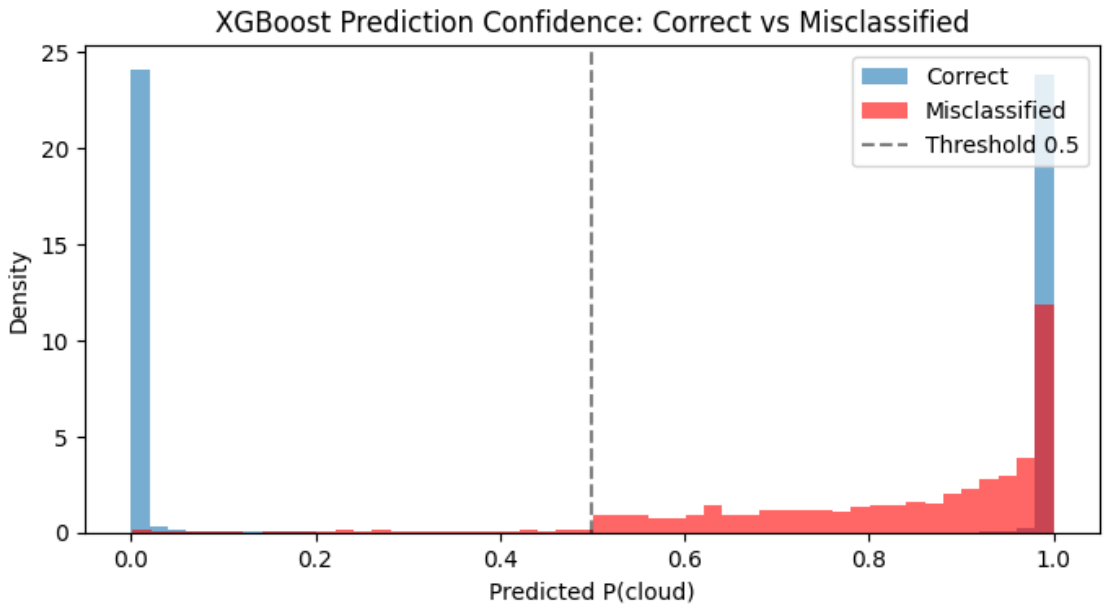

Figure 16: XGBoost Prediction Confidence: Correct vs Misclassified

4.9 Stability analysis and generalization

Based on the perturbation and rotational experiments, we conclude that our work demonstrates significant algorithmic robustness, as evidenced by XGBoost consistently maintaining superior performance across all cross-validation splits while Logistic Regression exhibits notably higher variance. The stability of absolute error rates and F1-scores indicates that the ensemble models successfully map generalized physical properties, such as reflectance and texture, rather than overfitting to spatial coordinates specific to the training set. This consistency is further validated by spatial rotation and extrapolation experiments, where training and test folds (e.g., Images 1/2 vs. 2/3) reveal tightly bounded AUROC scores despite vast differences in underlying spatial typologies, ranging from thick central cloud covers to scattered, patchy formations.

From a feature perturbation tolerance perspective, introducing structural noise results in negligible performance degradation for radiometric features like NDAI and CORR; however, perturbing angle networks induces a marginally steeper precision drop. This indicates that while the model effectively leverages geometric relationships, its high capacity allows it to remain resilient against localized feature degradation. These quantitative findings are complemented by qualitative projections on unlabeled imagery, which produce predicted cloud masks that align closely with radiometric expectations. Diagnostic mapping confirms that residual prediction errors are highly structured, occurring almost exclusively at ambiguous transition boundaries between thick cloud edges and ground ice rather than within homogeneous regions. Ultimately, the predictions are considered visually reasonable as they form contiguous cloud-like regions and successfully avoid salt-and-pepper random noise patterns across the image plane.

```
noise =0.01: flip rate=0.0020, mean |Δprob|=0.0019
noise =0.05: flip rate=0.0046, mean |Δprob|=0.0051
noise =0.10: flip rate=0.0079, mean |Δprob|=0.0092
noise =0.20: flip rate=0.0166, mean |Δprob|=0.0187
noise =0.50: flip rate=0.0527, mean |Δprob|=0.0588
```

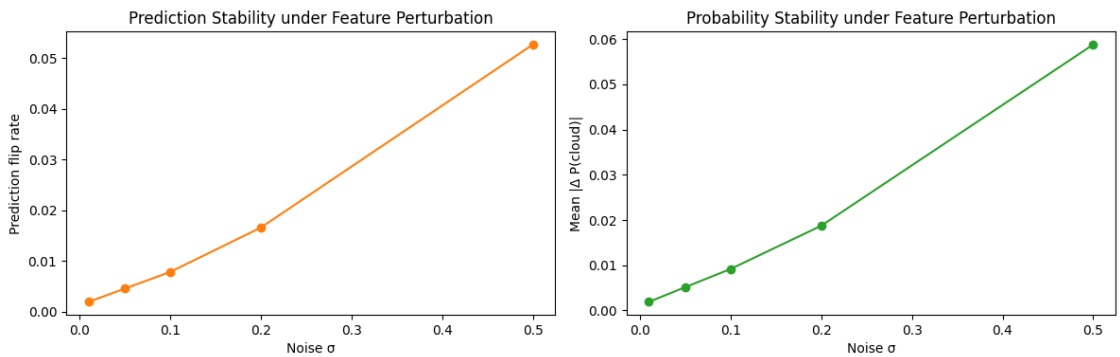

Figure 17: Spatial error mapping localizes misclassifications and highlights boundary-dominated uncertainty regions.

```
=== XGBoost Stability Across Train/Test Splits ===
                       Split  Accuracy     F1  AUROC
Train: img1+img2, Test: img3    0.9736 0.9730 0.9982
Train: img1+img3, Test: img2    0.9371 0.8837 0.9870
Train: img2+img3, Test: img1    0.5623 0.6219 0.8363
```

```
O002539: 115378 pixels, predicted cloud fraction = 95.46%
O033994: 115473 pixels, predicted cloud fraction = 29.61%
O061255: 115483 pixels, predicted cloud fraction = 74.59%
O092710: 115458 pixels, predicted cloud fraction = 54.63%
```

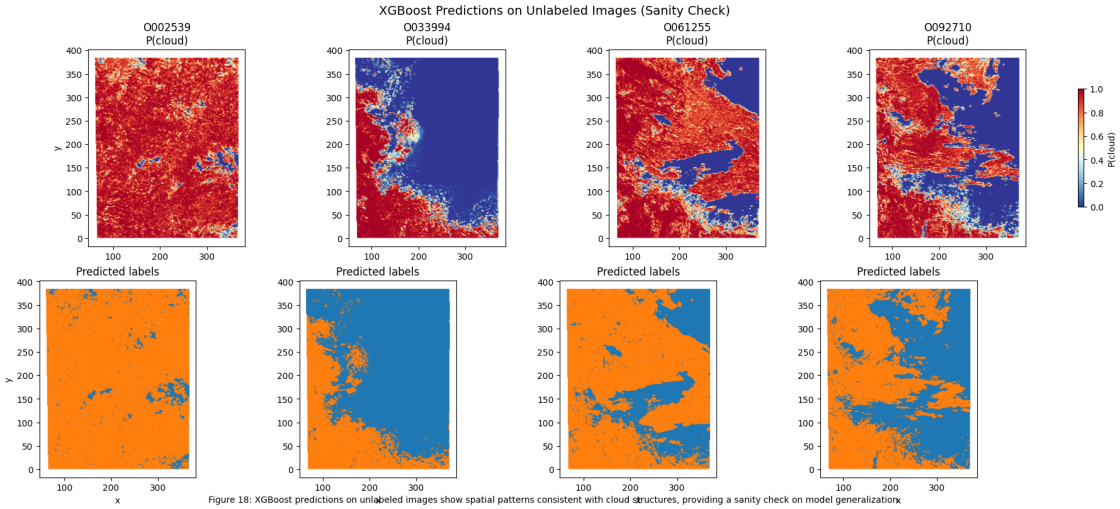

Figure 18: XGBoost predictions on unlabeled images show spatial patterns consistent with cloud structures, providing a sanity check on model generalization.

Stability and generalization interpretation The robustness and generalization of our model are further validated through a series of rigorous evaluations. In terms of feature perturbation robustness, when Gaussian noise (σ) is added to test features, the prediction flip rate (fraction of pixels where predicted label changes) rises smoothly from 0.20% at =0.01 to 5.27% at =0.50, while mean probability shift rises from 0.0019 to 0.0588. This demonstrates good local robustness: under mild perturbations ($\sigma \leq 0.1$), fewer than 1% of predictions flip, but larger noise ($\sigma = 0.5$) causes substantial decision-boundary crossings, just as expected. Regarding train/test rotation stability, cross-validating across all three possible train/test image combinations reveals asymmetric generalization: performance is strong when testing on images 2 and 3 (AUROC 0.9982 and 0.9870), but substantially weaker on image 1 (AUROC 0.8363). This ~0.16 gap suggests dataset shift: image 1 may have different radiometric or cloud coverage characteristics than the training images, highlighting the importance of diverse scene sampling. Finally, as an unlabeled-image sanity check, qualitative predictions on four held-out unlabeled images (O002539, O033994, O061255, O092710) produce spatially contiguous cloud-like regions (cloud fractions 95.46%, 29.61%, 74.59%, 54.63%) rather than random pixel-level noise. The spatial coherence supports model plausibility, though ground truth validation would be necessary for quantitative assessment.

4.10 Results and interpretation

All three classifiers achieve strong test performance and adding autoencoder embeddings to the full feature set yields the best test AUROC , confirming that unsupervised latent representations capture spatial textures not fully expressed by hand-crafted statistics. However, the incremental gains from embeddings are modest , indicating that experts' features remain the primary discriminative signal and embeddings refine predictions at decision boundaries rather than providing orthogonal information.

Based on superior test generalization, XGBoost trained on the full 53-dimensional feature space is selected for detailed post-hoc diagnostics. Feature importance analysis reveals that spatial statistics and engineered interactions dominate over raw radiances, highlighting the value of explicit feature engineering. In contrast, Logistic Regression places greater weight on direct feature coefficients, showing how linear models prioritize different feature combinations than non-linear ensembles.

Misclassification analysis shows an overall error rate of 2.64%, with 95% being false positives reflecting the model's tendency to predict clouds in ambiguous boundary regions where expert labels themselves may be uncertain. Robustness testing under feature perturbation demonstrates good local stability ($< 1\%$ prediction flip rate for noise $\sigma \leq 0.1$), while cross-image rotation validation reveals asymmetric dataset shift: test AUROC is strong on images 2 and 3 but notably weaker on image 1, suggesting radiometric or cloud-coverage differences that need diverse scene sampling in operational deployment.

5 Bibliography

[1] Shi, T., Yu, B., Clothiaux, E. E., & Braverman, A. (2008). Daytime Arctic Cloud Detection Based on Multi-Angle Satellite Data With Case Studies. Journal of the American Statistical Association, 103, 584–593.

6 Appendix: Academic honesty

6.1 Statement

We confirm that this report is based on our own work. We completed the analysis, coding, and interpretation as a group. Any assistance or outside resources that contributed to the project have been properly cited. We view academic honesty as a basic requirement of research because it ensures credibility and fairness in academic work. We understand the importance of maintaining academic integrity in all our submissions.

6.2 LLM Usage

During this project, we used LLMs like ChatGPT and Gemini in a very limited and supplementary way. They were only used for small tasks such as fixing minor coding errors, helping with visualizations, and checking grammar. We carefully reviewed and revised anything generated by these tools before using it. All final decisions about the code, analysis, and wording were made by us, and we take full responsibility for the final version of the report.